

1.7 MATRICES

Matrices are the R objects in which the elements of the same atomic type are arranged in a two-dimensional rectangular layout. Though programmers can create a matrix containing only characters or only logical values, they are not of much use. Programmers use matrices containing numeric elements to be used in mathematical calculations.

The basic syntax for creating a matrix in R is

```
matrix(data, nrow, ncol, byrow, dimnames)
```

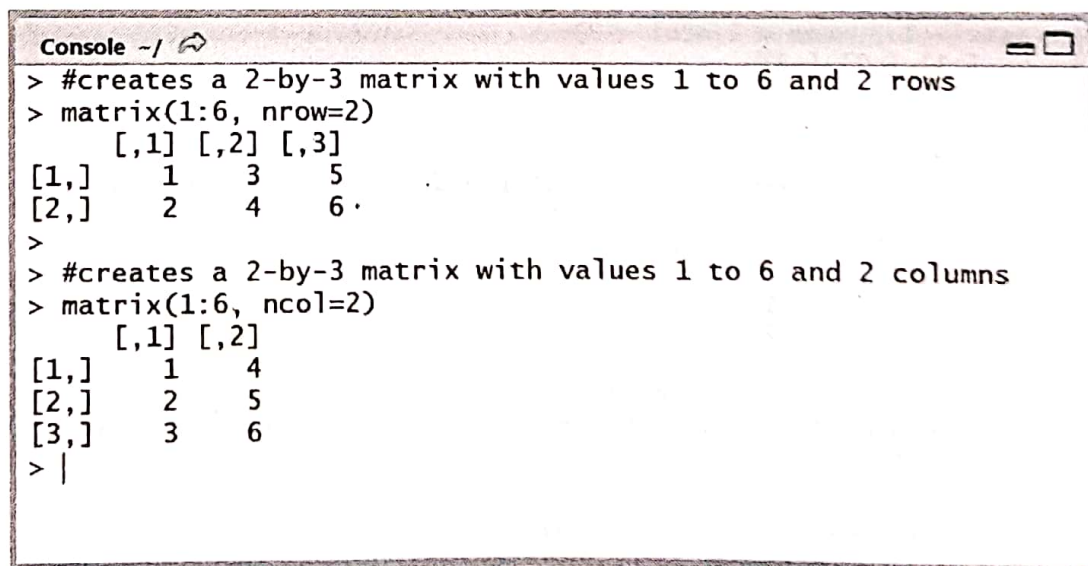
The following are the attributes used in matrix

1. *data* is the input vector which becomes the data elements of the matrix.
2. *nrow* is the number of rows to be created.
3. *ncol* is the number of columns to be created.
4. *byrow* is a logical clue. If TRUE then the input vector elements are arranged by row.
5. *dimnames* are the names assigned to the rows and columns.

1.7.1 Creating and Naming Matrices

As with vectors, a matrix can contain only one atomic vector type. This means that users can't have logicals and numerals in a matrix. It's a natural extension of the vector, going from one to two dimensions. Start with simply creating and naming a matrix. To build a matrix, users use the *matrix* function. Most importantly, it needs a vector containing the values that users want to place in the matrix, and at least one matrix dimension. Users can choose to specify the number of rows or columns.

Figure 1.48 shows the creation of a 2-by-3 matrix containing the values 1 to 6, by specifying the vector and setting the *nrow* argument to 2.



```

Console ~/
> #creates a 2-by-3 matrix with values 1 to 6 and 2 rows
> matrix(1:6, nrow=2)
      [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
>
> #creates a 2-by-3 matrix with values 1 to 6 and 2 columns
> matrix(1:6, ncol=2)
      [,1] [,2]
[1,]    1    4
[2,]    2    5
[3,]    3    6
> |
  
```

Fig. 1.48 Creating and naming a matrix

R sees that the input vector has a length 6 and that there have to be two rows. It then infers that such an arrangement will need 3 columns, such that the number of matrix elements matches the number of input vector elements. Users could just as well specify *ncol* instead of *nrow*; in this case, R infers the number of rows automatically.

In both these examples, R takes the vector containing the values 1 to 6, and fills it up, column by column. If users prefer to fill up the matrix in a row-wise fashion, such that 1, 2, and 3 are in the first row, users can set the *byrow* argument of matrix to 'TRUE' as shown in Fig. 1.49.

```

Console - / 
> #Fill up the matrix in row wise fashion
> matrix(1:6,nrow=2,byrow = TRUE)
      [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
> |

```

Fig. 1.49 Matrix with byrow argument set to TRUE

Remember how R did recycling when we were subsetting vectors using logical vectors? The same thing happens when we pass a vector that is too short to fill up the entire matrix to the matrix function. Suppose we pass a vector containing the values 1 to 3 to the matrix function, and explicitly say that we want a matrix with 2 rows and 3 columns; R fills up the matrix column by column and simply repeats the vector. If we want to fill up the matrix with a vector whose multiple does not fit perfectly in the matrix (for example when we want to pass a 4-element vector in a 6-element matrix), R generates a warning message as shown in Fig. 1.50.

```

Console - / 
> #Vector containing the values 1-3 to the matrix function, with 2
rows and 3 columns
> matrix(1:3,nrow = 2,ncol = 3)
      [,1] [,2] [,3]
[1,]    1    3    2
[2,]    2    1    3
>
> #matrix with a vector whose multiple does not nicely fit in the
matrix
> matrix(1:4,nrow = 2,ncol = 3)
      [,1] [,2] [,3]
[1,]    1    3    1
[2,]    2    4    2
warning message:
In matrix(1:4, nrow = 2, ncol = 3) :
  data length [4] is not a sub-multiple or multiple of the number
of columns [3]
> |

```

Fig. 1.50 Error message when passing a vector whose multiple does not fit in the matrix

Apart from the *matrix()* function, there is yet another easy way to create matrices that is more intuitive in some cases. Users can paste vectors together using the *cbind()* and *rbind()* functions. *cbind()*, short for column bind, takes the vectors passed and sticks them together as if they were columns of a matrix. The *rbind()* function, short for row bind, does the same thing but takes the input as rows and makes a matrix out of them. They're often easier to use than the *matrix()* function. Suppose we have a matrix 'm', containing the elements 1 to 6. If we want to add another row containing the values 7, 8, 9 to it, we can do this with *cbind()* function as shown in Fig. 1.51.


```

Console /
> cbind(1:3,1:3)
      [,1] [,2]
[1,]    1    1
[2,]    2    2
[3,]    3    3
>
> rbind(1:3,1:3)
      [,1] [,2] [,3]
[1,]    1    2    3
[2,]    1    2    3
>
> #matrix 'm', containing the elements 1 to 6
> m<- matrix(1:6,byrow = TRUE,nrow = 2)
>
> #Add 7,8,9 values using rbind function
> rbind(m,7:9)
      [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
[3,]    7    8    9
>
> #Add 10,11 values using rbind function
> cbind(m,c(10,11))
      [,1] [,2] [,3] [,4]
[1,]    1    2    3   10
[2,]    4    5    6   11
> |

```

Fig. 1.51 cbind() function

Next up is naming the matrix. In the case of vectors, we simply used the *names()* function, but in the case of matrices, we can assign names to both columns and rows. That's why R came up with the *rownames()* and *colnames()* functions as shown in Fig. 1.52.

```

Console /
> #matrix 'm', containing the elements 1 to 6
> m<- matrix(1:6,byrow = TRUE,nrow = 2)
>
> rownames(m)<- c("row1","row2")
> m
      [,1] [,2] [,3]
row1    1    2    3
row2    4    5    6
>
> colnames(m)<- c("col1","col2","col3")
> m
      col1 col2 col3
row1    1    2    3
row2    4    5    6
> |

```

Fig. 1.52 Naming columns and rows of a matrix

Setting the column names with a vector of length 3 gives us a fully named matrix. Just as with vectors, there are also one-liner ways of naming matrices while we are creating it. We can use the *dimnames* argument of the *matrix* function for this. We need to specify a list that has a vector of row names as the first element and a vector of column names as the second element as shown in Fig. 1.53.

```

Console -/
> #matrix 'm', containing the elements 1 to 6
> m<- matrix(1:6,byrow = TRUE,nrow = 2, dimnames = list(c("row1",
"row2"),c("col1","col2","col3")))
>
> #Print m
> m
      col1 col2 col3
row1    1    2    3
row2    4    5    6
> |

```

Fig. 1.53 Naming rows and columns using *dimnames* attribute

Coercion of Matrix Elements

If we try to store different data types in a matrix, coercion automatically takes place. Have a look at the two matrices in Fig. 1.54—one containing numerics and the other one containing characters. These two matrices are bound together in a column-wise fashion using *cbind()*. The numeric matrix elements are coerced to characters to end up with a matrix that contains only characters. To have a multi-dimensional data structure that can contain different elements, we will have to use the lists or more specifically *data frames*.

```

Console -/
> num <- matrix(1:8,ncol = 2)
> num
      [,1] [,2]
[1,]    1    5
[2,]    2    6
[3,]    3    7
[4,]    4    8
>
> char <- matrix(LETTERS[1:6], nrow = 4, ncol = 3)
> char
      [,1] [,2] [,3]
[1,] "A"  "E"  "C"
[2,] "B"  "F"  "D"
[3,] "C"  "A"  "E"
[4,] "D"  "B"  "F"
>
> cbind(num,char)
      [,1] [,2] [,3] [,4] [,5]
[1,] "1"  "5"  "A"  "E"  "C"
[2,] "2"  "6"  "B"  "F"  "D"
[3,] "3"  "7"  "C"  "A"  "E"
[4,] "4"  "8"  "D"  "B"  "F"
> |

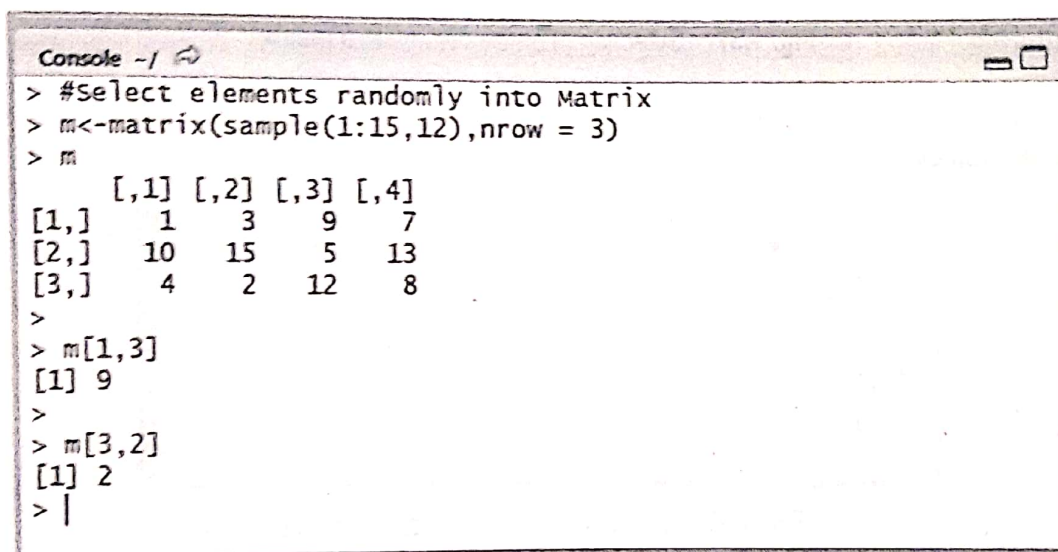
```

Fig. 1.54 Automatic coercion of different data types

1.7.2 Matrix Subsetting

Just as in the case of vectors, there are situations in which we want to select a single element or entire parts of a matrix to continue the analysis with. Again, we can use square brackets for this, but the fact that we are dealing with two dimensions now, complicates things a bit. Have a look at the matrix in Fig. 1.55 containing some random numbers. If we want to select a single element from this matrix, we will have to specify both the row and the column of the element of interest. If we want to select the number 15, located at the first row and the third column we type `m[1,3]`.

As is evident, the first index refers to the row and the second one refers to the column. Likewise, to select the number 1, at row 3 and column 2, we write `m[3,2]`. Notice that the results are single values, that is, vectors of length 1.



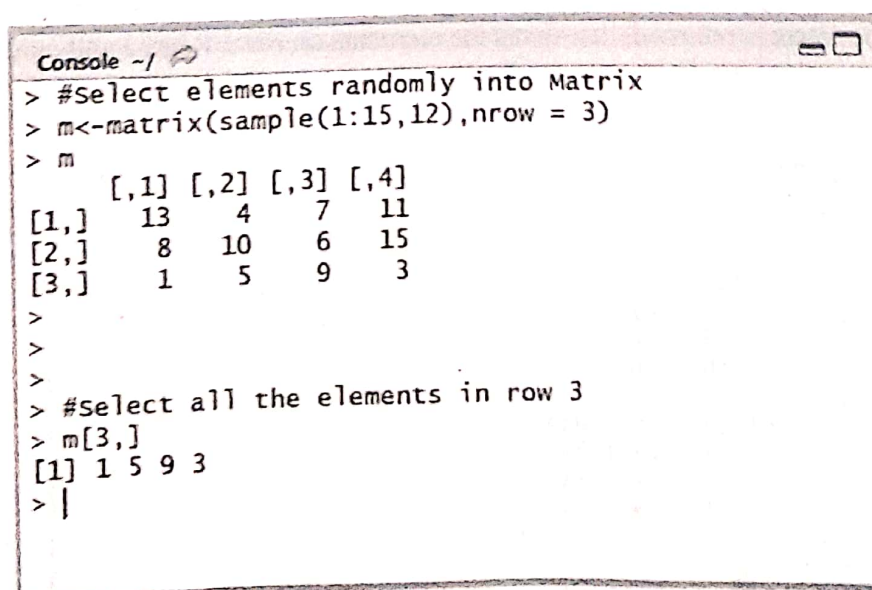
```

Console ~/
> #Select elements randomly into Matrix
> m<-matrix(sample(1:15,12),nrow = 3)
> m
      [,1] [,2] [,3] [,4]
[1,]    1    3    9    7
[2,]   10   15    5   13
[3,]    4    2   12    8
>
> m[1,3]
[1] 9
>
> m[3,2]
[1] 2
> |

```

Fig. 1.55 Vectors of length 1

Now, what if we want to select an entire row or column from this matrix? We can do this by missing either the row or column index within the square brackets. Instead of writing `[3, 2]` to select the element at row 3 and column 2, if we write `[3,]`, this would select all the elements that are in row 3, namely 1, 5, 9, and 3 as shown in Fig. 1.56.



```

Console ~/
> #Select elements randomly into Matrix
> m<-matrix(sample(1:15,12),nrow = 3)
> m
      [,1] [,2] [,3] [,4]
[1,]   13    4    7   11
[2,]    8   10    6   15
[3,]    1    5    9    3
>
>
>
> #Select all the elements in row 3
> m[3,]
[1] 1 5 9 3
> |

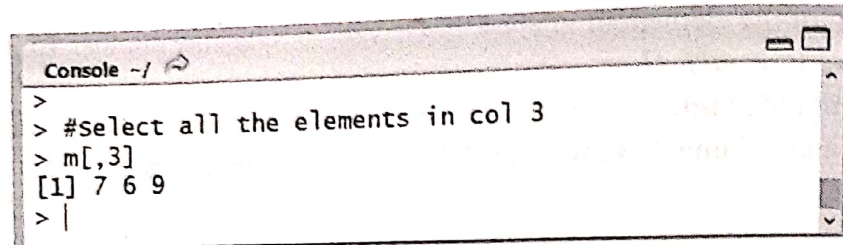
```

Fig. 1.56 Selection of entire row in a matrix

34 Statistical Programming in R

Notice here that the result is not a matrix anymore! It's also a vector, but this time one that contains more than 1 element. We selected a single row from the matrix so a vector is sufficient to store this one-dimensional information. To select columns, we can work similarly, but this time the first index should be removed.

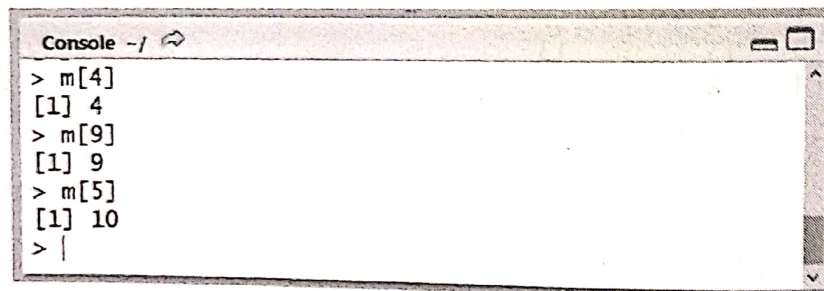
To select the entire third column, $m[,3]$ is used. Again, a vector results, this time of length 3, corresponding to the third column of m as shown in Fig. 1.57.



```
Console ~/
>
> #select all the elements in col 3
> m[,3]
[1] 7 6 9
> |
```

Fig. 1.57 Selection of entire column in a matrix

Figure 1.58 illustrates what happens when we decide not to include a comma to clearly discern between column and row indices.

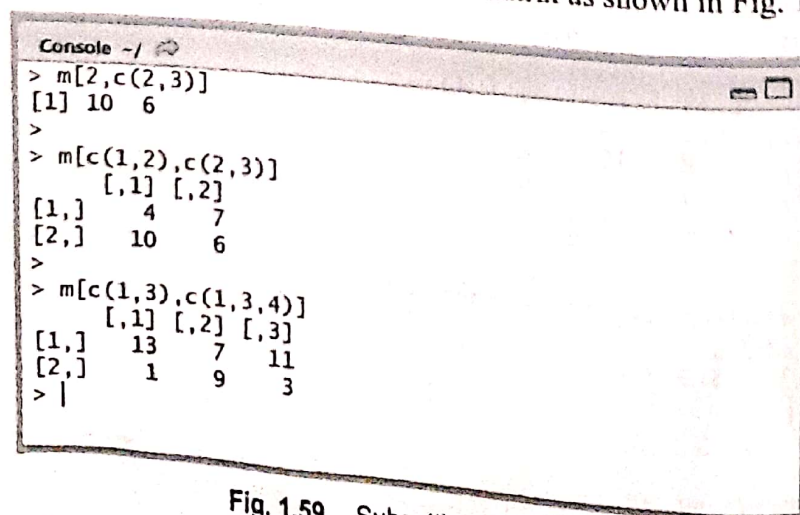


```
Console ~/
> m[4]
[1] 4
> m[9]
[1] 9
> m[5]
[1] 10
> |
```

Fig. 1.58 Selection without including comma

Well, when we pass a single index to subset a matrix, R simply goes through the matrix, column by column, from left to right. If we pass $m[9]$, we should get 9, in the third row and third column. In vector subsetting, we also learned how to select multiple elements. In matrices also, this is of course possible and the principles are just the same. Say, for example, we want to select the values 10 and 6, from the middle of the matrix. Again, the result is a vector, because one dimension suffices.

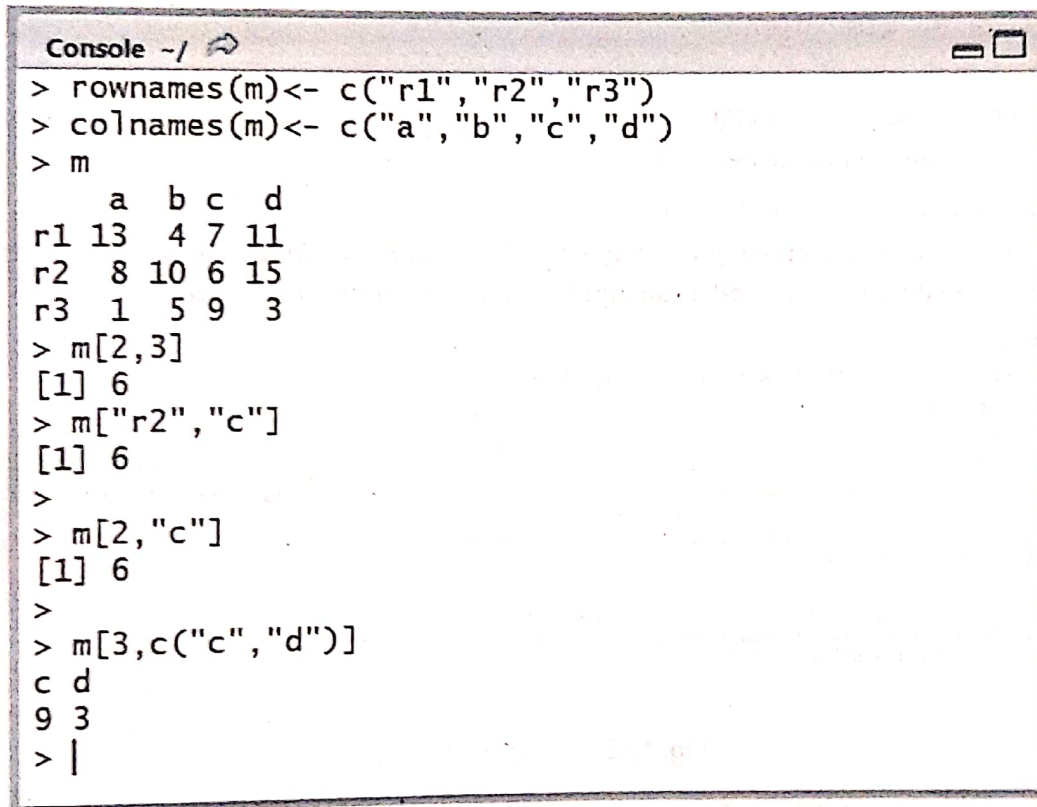
However, we can't select elements that don't have either the row or the column index in common. If we want to select 4 from row 1 and column 2, and 6 from row 2 and column 3, this call will not give the desired result. Instead, a sub-matrix is returned, that spans the elements on rows 1 and 2 and columns 2 and 3. These sub-matrices can also be built up from disjoint places in the matrix as shown in Fig. 1.59



```
Console ~/
> m[2,c(2,3)]
[1] 10 6
>
> m[c(1,2),c(2,3)]
      [,1] [,2]
[1,]    4    7
[2,]   10    6
>
> m[c(1,3),c(1,3,4)]
      [,1] [,2] [,3]
[1,]   13    7   11
[2,]    1    9    3
> |
```

Fig. 1.59 Subsetting matrices

Let us now take a look at subsetting by names first, though, we will have to name the matrix. In fact, subsetting by names works exactly the same as by index, where users just replace the indices with the corresponding names. To select 6, we can use the row index 2 and column index 3, or use the row name *r2* and column name *c3*. We can even use a combination of both. Just remember to surround the row and column names with quotes as shown in Fig. 1.60. Selecting multiple elements and sub-matrices from a matrix is straightforward as well. To select elements on row *r3* and in the last two columns, we can use the function shown in Fig. 1.60.



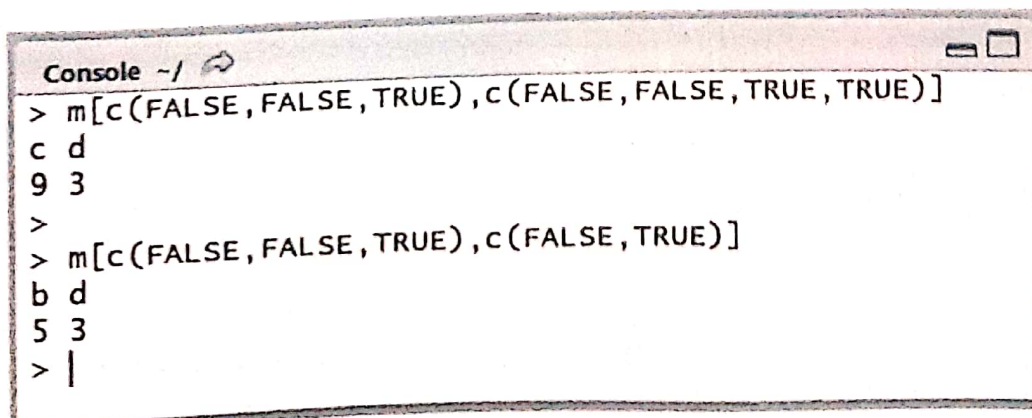
```

Console ~/
> rownames(m) <- c("r1", "r2", "r3")
> colnames(m) <- c("a", "b", "c", "d")
> m
      a  b  c  d
r1 13  4  7 11
r2  8 10  6 15
r3  1  5  9  3
> m[2,3]
[1] 6
> m["r2", "c"]
[1] 6
>
> m[2, "c"]
[1] 6
>
> m[3, c("c", "d")]
c d
9 3
> |

```

Fig. 1.60 Selecting multiple elements and sub-matrices from a matrix

Finally, we can also use logical vectors; rows and columns corresponding to *TRUE* are returned as results, whereas those corresponding to *FALSE* are left out. To select the same elements as in the previous call, we can use the matrix *m* as reference. The rules of vector recycling also apply here. Suppose that we only pass a vector of length 2 to perform a selection on the columns. The column selection vector gets recycled to *FALSE*, *TRUE*, *FALSE*, *TRUE*, giving the same result as shown in Fig. 1.61.



```

Console ~/
> m[c(FALSE, FALSE, TRUE), c(FALSE, FALSE, TRUE, TRUE)]
c d
9 3
>
> m[c(FALSE, FALSE, TRUE), c(FALSE, TRUE)]
b d
5 3
> |

```

Fig. 1.61 Recycling vectors

1.8 ARRAYS

An array is a collection of similar data of the same type, for example, numeric. R provides commands for easy design and management of arrays, and it is a special case of matrix formation. R uses a vector to create an array only if there is a dimension vector with `dim` attributes.

The `array()` function is used to create an array by passing vectors as input and by using the values in the `dim` parameter.

The basic syntax for creating an array in R is

```
array(data, dim, dimname)
```

where,

- `data` is the input values to the vector
- `dim` is used to create the dimension
- `dimname` is used to assign a name to `dim`

Figure 1.62 shows a program creating an array of 3 elements. First, values are assigned to the object `vec`. Then an array is created with `vec` passed as an input. Finally the array is displayed.

```

RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Go to file/function
Addins
Source on Save
Run Source
1 vec <- c(5,9,3) #assigning input to vector vec
2 result <- array(c(vec)) #passing vec as a input to the array.
3 print(result) # print result
4
> vec <- c(5,9,3) #assigning input to vector vec
> result <- array(c(vec)) #passing vec as a input to the array.
> print(result) # print result
[1] 5 9 3
>

```

Fig. 1.62 Creating an array

Example 1.15 Write a program to create an array of 3×3 matrixes with 3 rows and 3 columns.

Solution:

Figure 1.63 shows the code to create an array of matrices. First values are assigned to objects `vec1` and `vec2`. Then the array is created and displayed in matrix form.

```

RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Go to file/function
Addins
Source on Save
Run Source
1 vec1 <- c(5,9,3) #assigning values to vec1
2 vec2 <- c(10,11,12,13,14,15) #assigning values to vec2
3 result <- array(c(vec1,vec2),dim = c(3,3,2)) # for creating array
4 print(result) #display result
... 1
      [,1] [,2] [,3]
[1,]    5   10   13
[2,]    9   11   14
[3,]    3   12   15
... 2
      [,1] [,2] [,3]
[1,]    5   10   13
[2,]    9   11   14
[3,]    3   12   15

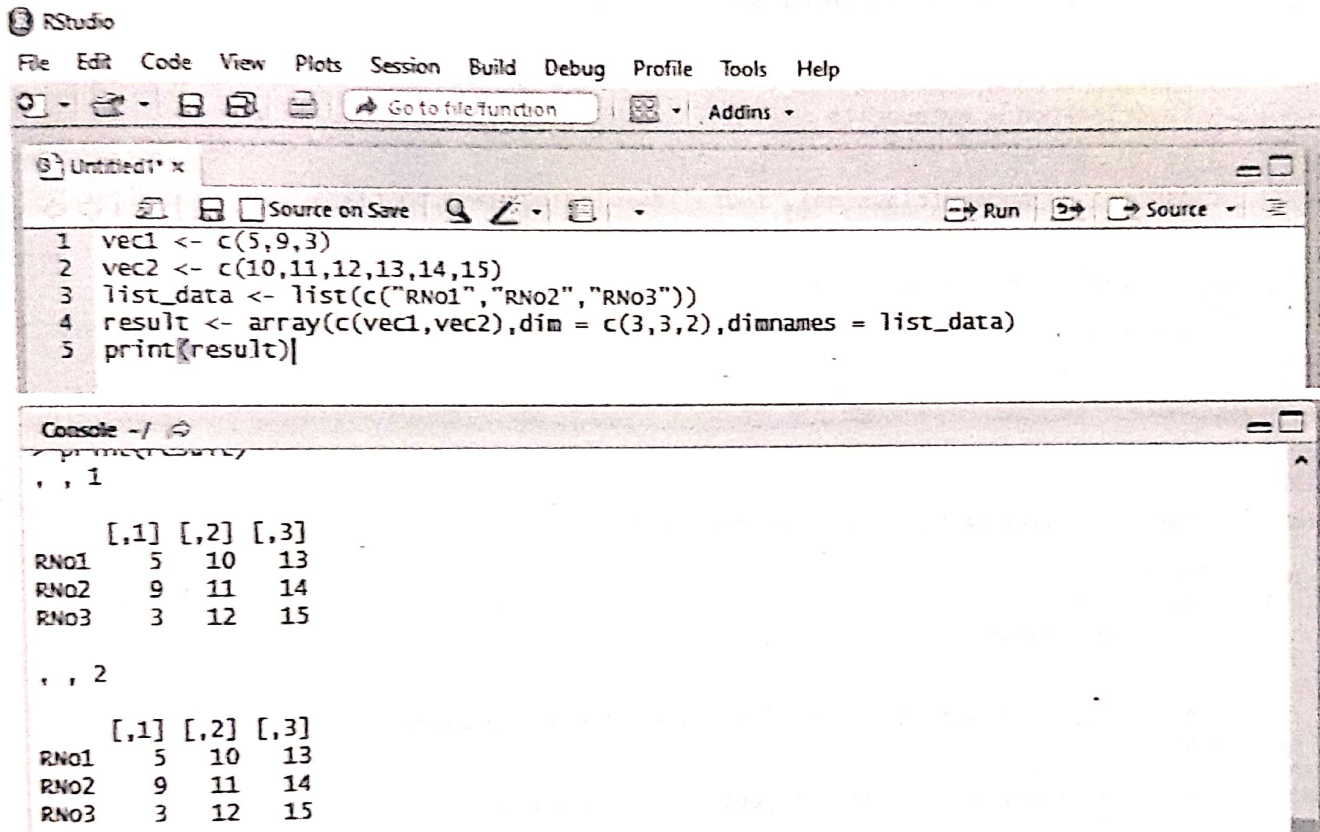
```

Fig. 1.63 Creation of array of matrices

Example 1.16 Write a program to assign dimension name to an array of 3×3 matrixes.

Solution:

Refer to Figure 1.64. First, values are assigned to objects *vec1* and *vec2*. Then an array is created and names *RNo1*, *RNo2*, and *RNo3* are assigned to the array matrix. Finally the dimension names are displayed in a matrix form.



```

RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Go to file/function Addins
Untitled1.R
Source on Save Run Source
1 vec1 <- c(5,9,3)
2 vec2 <- c(10,11,12,13,14,15)
3 list_data <- list(c("RNo1","RNo2","RNo3"))
4 result <- array(c(vec1,vec2),dim = c(3,3,2),dimnames = list_data)
5 print(result)

Console -/
print(result)
, , 1
  [,1] [,2] [,3]
RNo1   5   10   13
RNo2   9   11   14
RNo3   3   12   15

, , 2
  [,1] [,2] [,3]
RNo1   5   10   13
RNo2   9   11   14
RNo3   3   12   15
  
```

Fig. 1.64 Assigning dimension names to array of matrices

1.9 CLASS

Everything in R language is considered as an object and every object is associated with a class. We can create a class in R using the *class* keyword. In R there are different types of classes; the S3 classes are discussed in this section.

1.9.1 S3 Class

S3 classes are used to implement an object-oriented (OO) concept known as generic function OO. In S3 class, we use the concept of list property, where a list is created and a list class is set and returned. Here a function is used to make a list and the data defined in a list track by class.

Example 1.17 Write a program to create a class, object, and function.

Solution:

Figure 1.65 shows the code to create a class.

- Line number 1 is used to assign values to function parameters.
- Lines 3–5 are used to create a list of items.
- Lines 11 and 15 are used to create the object.